ICT2216/ICT2516C
Secure Software Development

Secure Software Design x01

Raymond Chan and Tram Truong Huu

Before we start... (11 Sept. 2023 event)

- Banana Token Hack
  - Price falls from $8.70 to $0.02 in less than 3 hours.

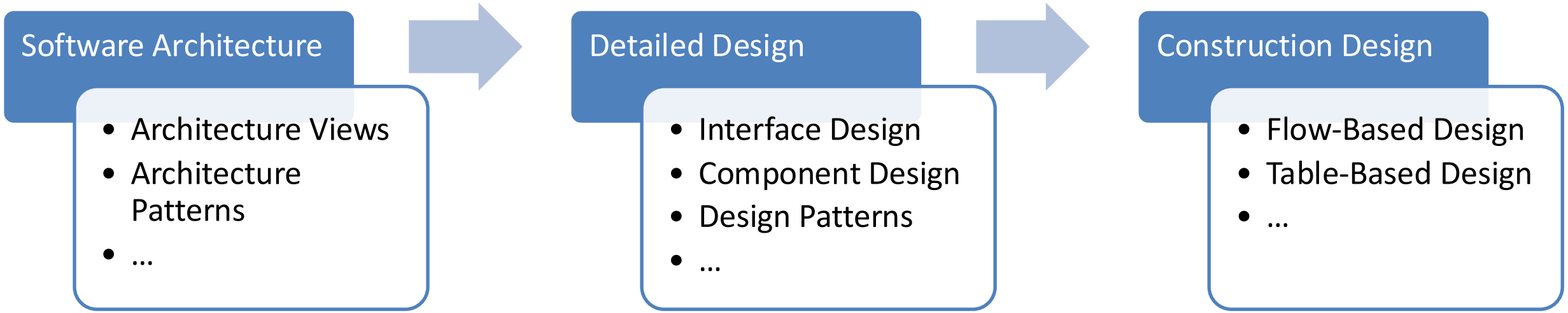

```
270         if (fees > 0) {
271             amount = amount - fees;
272             unchecked {
273                 _balances[address(this)] += fees;
274             }
275             emit Transfer(from, address(this), fees);
276         }
277     }
278
279     uint256 senderBalance = _balances[from];
280     require(senderBalance >= amount, "ERC20: transfer amount exceeds balance");
281     unchecked {
282         _balances[from] = senderBalance - amount;
283         _balances[to] += amount;
284     }
```

https://blog.solidityscan.com/banana-token-hack-analysis-3f6f84c08b8f

Agenda

- General Process of Software Design
- Security by Design Principles - OWASP
  - Security Architecture
  - Secure Software Design
  - Secure Agile Development

General Process of Software Design



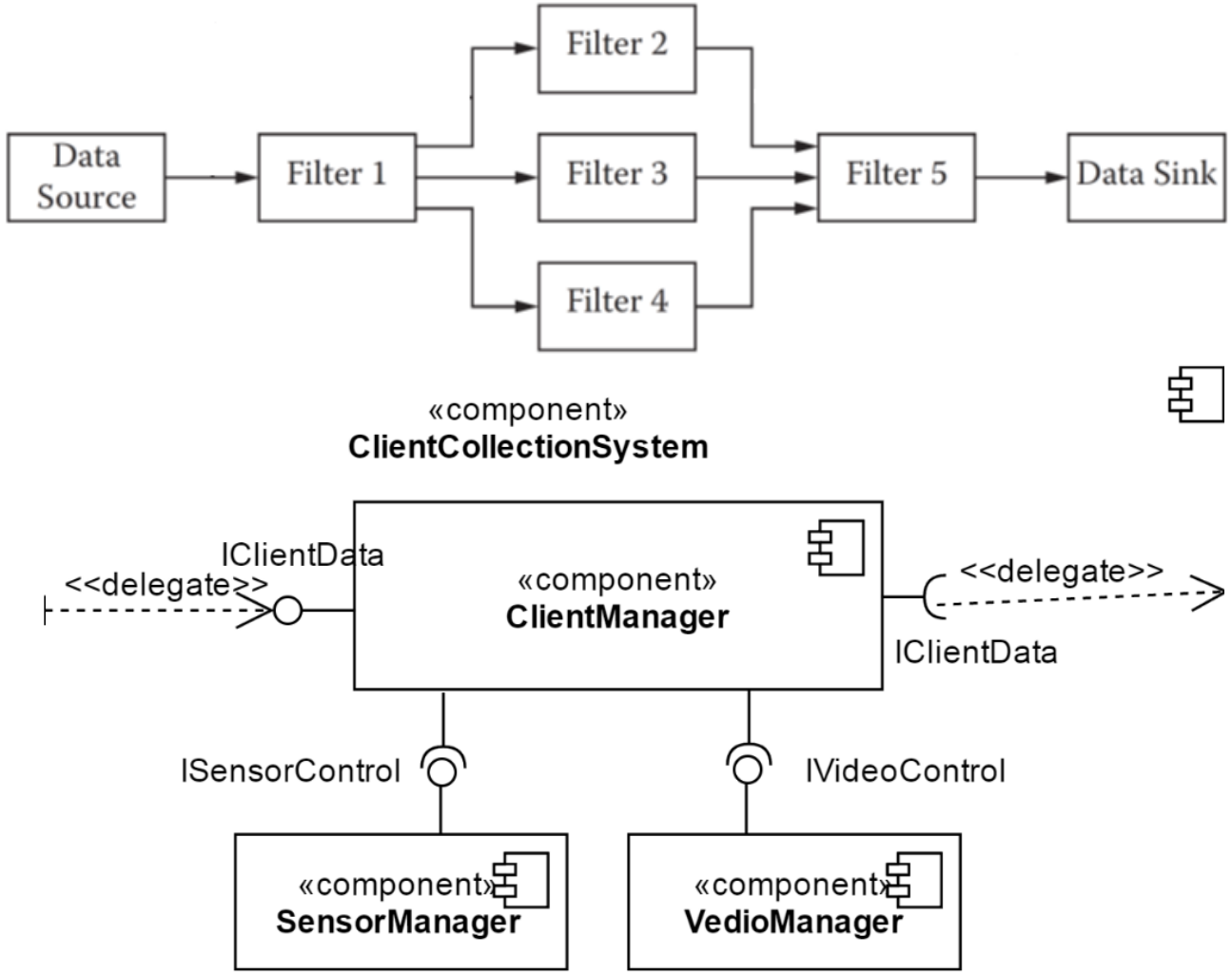
The detailed design activity builds on the software architecture to provide white-box design elements of the structure and behavior of the software.

Two major tasks:
Interface design refers to the design activity that deals with specification of interfaces between components in the design.

Component design refers to modeling the internal structure and behavior of components.
In object-oriented systems, the internal structure of components is typically modeled using UML through one or more diagrams, including class and sequence diagrams.

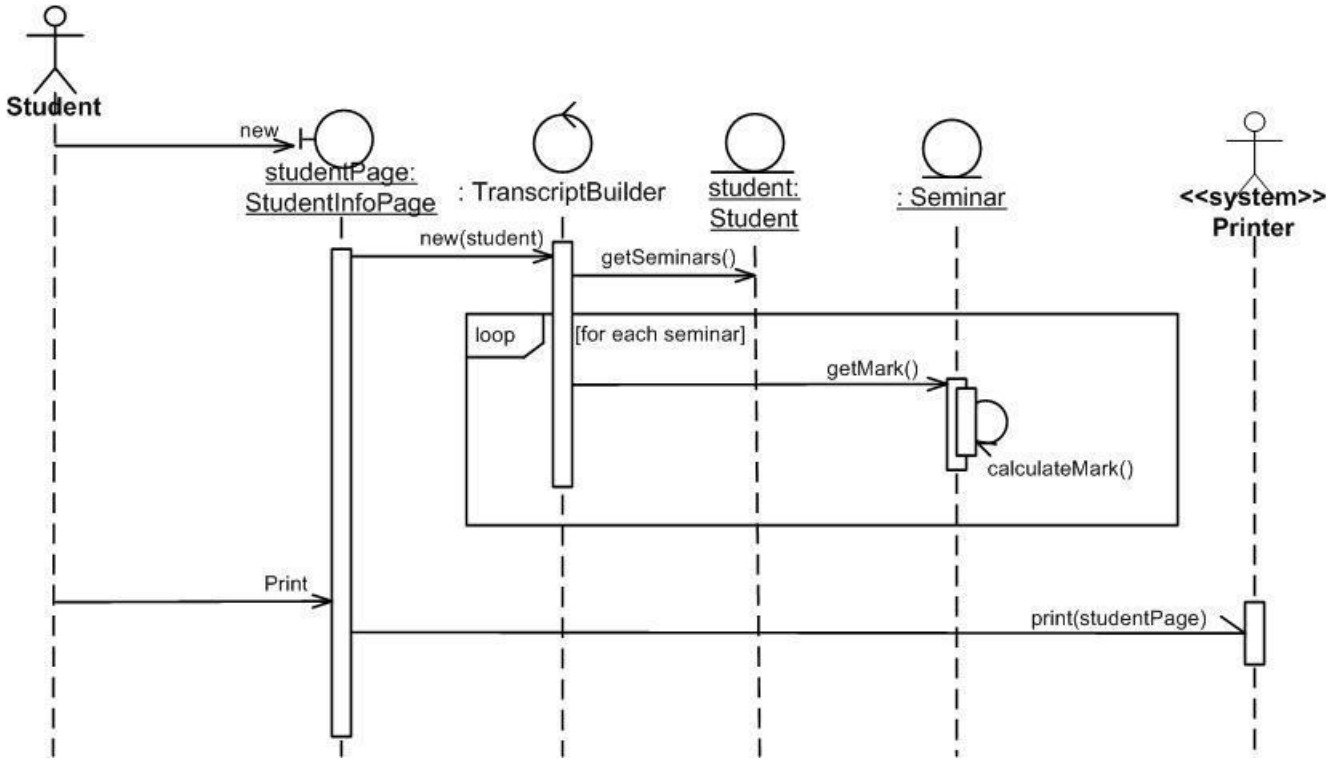
Recap

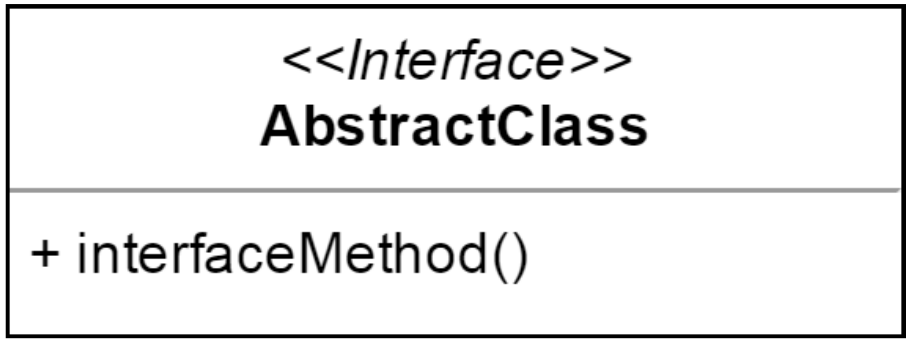
- Design Software Architecture
  - Using block-and-line Diagrams
  - Using Component Diagram
  - Using Styles and Patterns
  - Using Data Flow

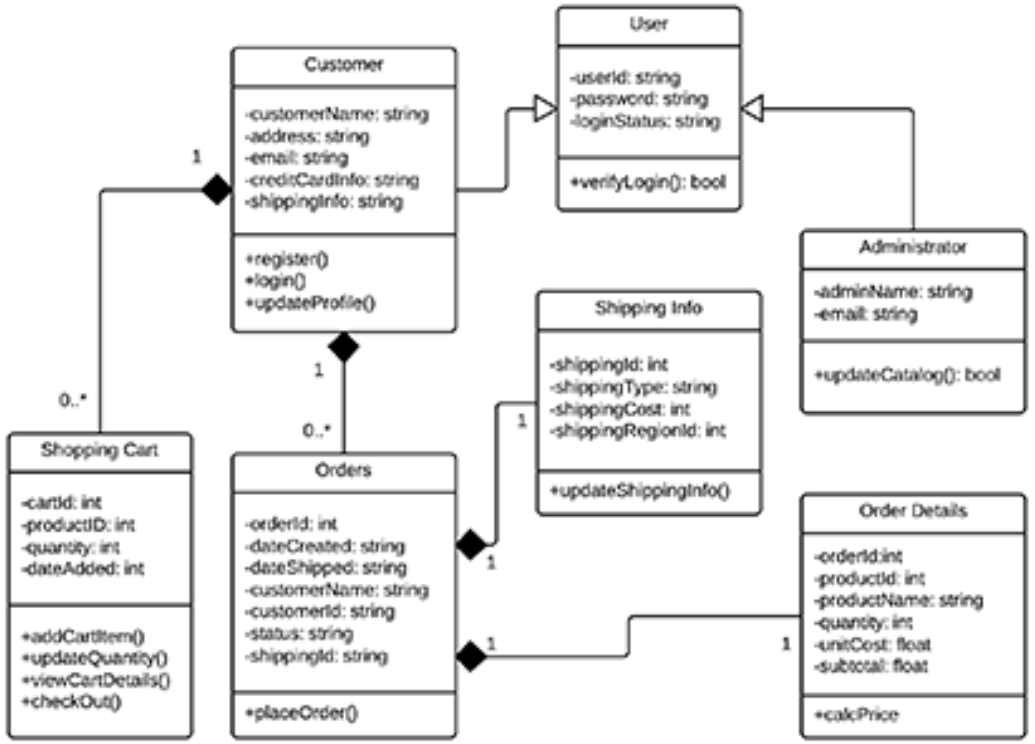


Recap

1. Interface Design
2. Designing Internal Structure of Components
3. Modeling Internal Behavior of Components







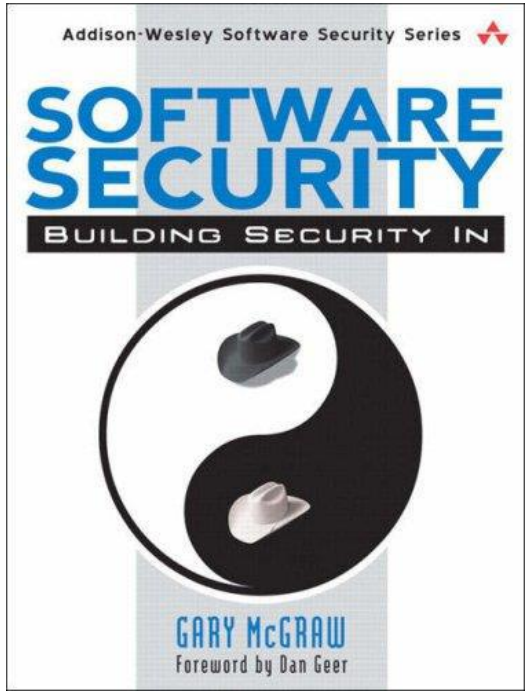
Recap: Making Secure Software

- Flawed Approach: Design and build software, and ignore security at first
  - Add security once the functional requirements are satisfied
- Better approach: Build security in from the start
  - Incorporate security-minded thinking into all phases of the development process

Previous Close	96.66	Market Cap	11.192B
Open	94.40	Beta	1.06
Bid	0.00 x 0	PE Ratio (TTM)	19.69
Ask	0.00 x 0	EPS (TTM)	4.72
Day's Range	90.72 - 95.69	Earnings Date	Oct 24, 2017 - Oct 30, 2017
52 Week Range	89.59 - 147.02	Dividend & Yield	1.56 (1.58%)
Volume	16,707,681	Ex-Dividend Date	2017-08-23
Avg. Volume	1,787,251	1y Target Est	136.92

Recap: Design Flaws

- Recall that software defects consist of both flaws and bugs
  - Flaws are problems in the design
  - Bugs are problems in the implementation
- We avoid flaws during the design phase
- Studies [1] show that 50% of security problems are flaws



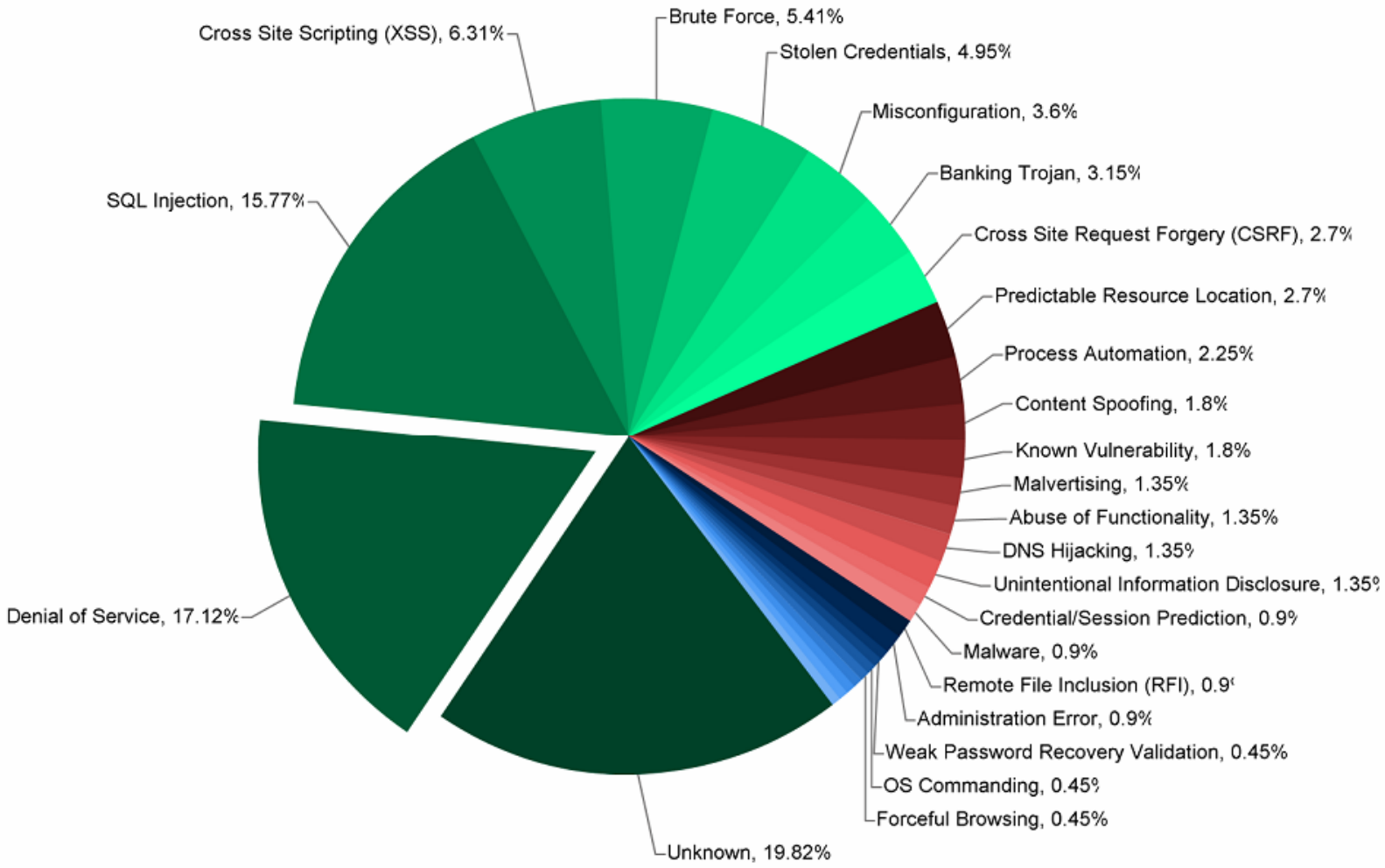
[1] Gary McGraw. 2006. Software Security: Building Security in. Addison-Wesley Professional.

Security by Design Principles - OWASP

- These security principles have been taken from the previous edition of the OWASP Development Guide and normalized with the security principles outlined in Howard and LeBlanc's excellent *Writing Secure Code*.
  1. Minimize attack surface area
  2. Establish secure defaults
  3. Principle of Least privilege
  4. Principle of Defense in depth
  5. Fail securely
  6. Don't trust services
  7. Separation of duties
  8. Avoid security by obscurity
  9. Keep security simple
  10. Fix security issues correctly

Minimize attack surface area

- **Every feature** that is added to an application **adds a certain amount of risk** to the overall application.
- The aim for secure development is to **reduce the overall risk** by **reducing the attack surface area**.
- When you create a new process / machine, attack surface WILL increase!



Minimize attack surface area: An example

- A web application implements **online help** with a **search function**. The search function may be vulnerable to SQL injection attacks:
  - If the help feature was **limited to authorized users**, the attack likelihood is reduced.
  - If the help feature's search function was **gated through centralized data validation routines**, the ability to perform SQL injection is dramatically reduced.
  - However, if the help feature was **re-written to eliminate the search function** (through better user interface, for example), this almost eliminates the attack surface area, even if the help feature was available to the Internet at large.

Attack Surface Analysis Cheat Sheet - OWASP

Type to search

CHEATSHEETS

Introduction

Index Alphabetical

Index ASVS

Index Proactive Controls

AJAX Security

Abuse Case

Access Control

Attack Surface Analysis

Authentication

Authorization Testing Automation

Bean Validation

C-Based Toolchain Hardening

C-Based Toolchain Hardening

Choosing and Using Security Questi...

Clickjacking Defense

Content Security Policy

Credential Stuffing Prevention

What is Attack Surface Analysis and Why is it Important?

This article describes a simple and pragmatic way of doing Attack Surface Analysis and managing an application's Attack Surface. It is targeted to be used by developers to understand and manage application security risks as they design and change an application, as well as by application security specialists doing a security risk assessment. The focus here is on protecting an application from external attack - it does not take into account attacks on the users or operators of the system (e.g. malware injection, social engineering attacks), and there is less focus on insider threats, although the principles remain the same. The internal attack surface is likely to be different to the external attack surface and some users may have a lot of access.

Attack Surface Analysis is about mapping out what parts of a system need to be reviewed and tested for security vulnerabilities. The point of Attack Surface Analysis is to understand the risk areas in an application, to make developers and security specialists aware of what parts of the application are open to attack, to find ways of minimizing this, and to notice when and how the Attack Surface changes and what this means from a risk perspective.

Attack Surface Analysis is usually done by security architects and pen testers. But developers should understand and monitor the Attack Surface as they design and build and change a system.

Attack Surface Analysis helps you to:

1. identify what functions and what parts of the system you need to review/test for security

Identifying and Mapping the Attack Surface

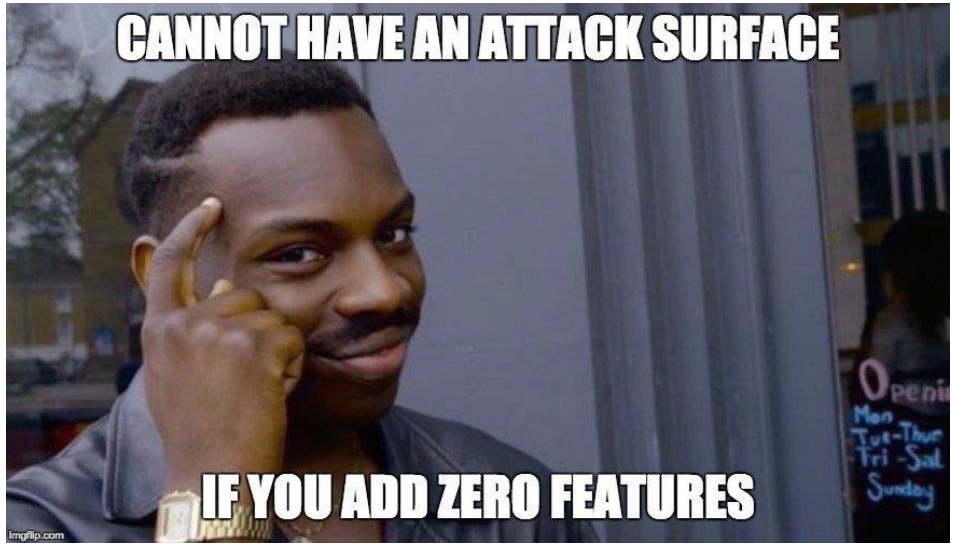
Read through the source code and identify different points of entry/exit:

- User interface (UI) forms and fields
- HTTP headers and cookies
- APIs
- Files
- Databases
- Other local storage
- Email or other kinds of messages
- Run-time arguments
- ...Your points of entry/exit

Identifying and Mapping the Attack Surface

The total number of different attack points can easily add up into the thousands or more. To make this manageable, break the model into **different types based on function, design and technology**:

- Login/authentication entry points
- Admin interfaces
- Inquiries and search functions
- Data entry (CRUD) forms
- Business workflows
- Transactional interfaces/APIs
- Operational command and monitoring interfaces/APIs
- Interfaces with other applications/systems
- …Your types



Establish secure defaults

- There are many ways to deliver an “out of the box” experience for users.
- However, by default, the experience should be secure, and it should be up to the user to reduce their security – if they are allowed.

- For example, **by default**, password aging and complexity should be enabled. **Users might be allowed to turn these two features off to simplify their use of the application at the cost of increase in their risk**.

Establish secure defaults

- HTTPS – by default
- All login attempt notification – by default
- Any information change notification – by default
- Ask users to reset password when 1st time login – by default

Principle of Least privilege

- The principle of **least privilege** recommends that accounts have the least amount of privilege required to perform their business processes.

- This encompasses user rights, resource permissions such as **CPU limits, memory, network, and file system permissions**.

Principle of Least privilege: An example

- If a middleware server only requires access to the network, read access to a database table, and the ability to write to a log, this describes **all the permissions** that should be granted.
- Under no circumstances should the middleware be granted **administrative privileges**.

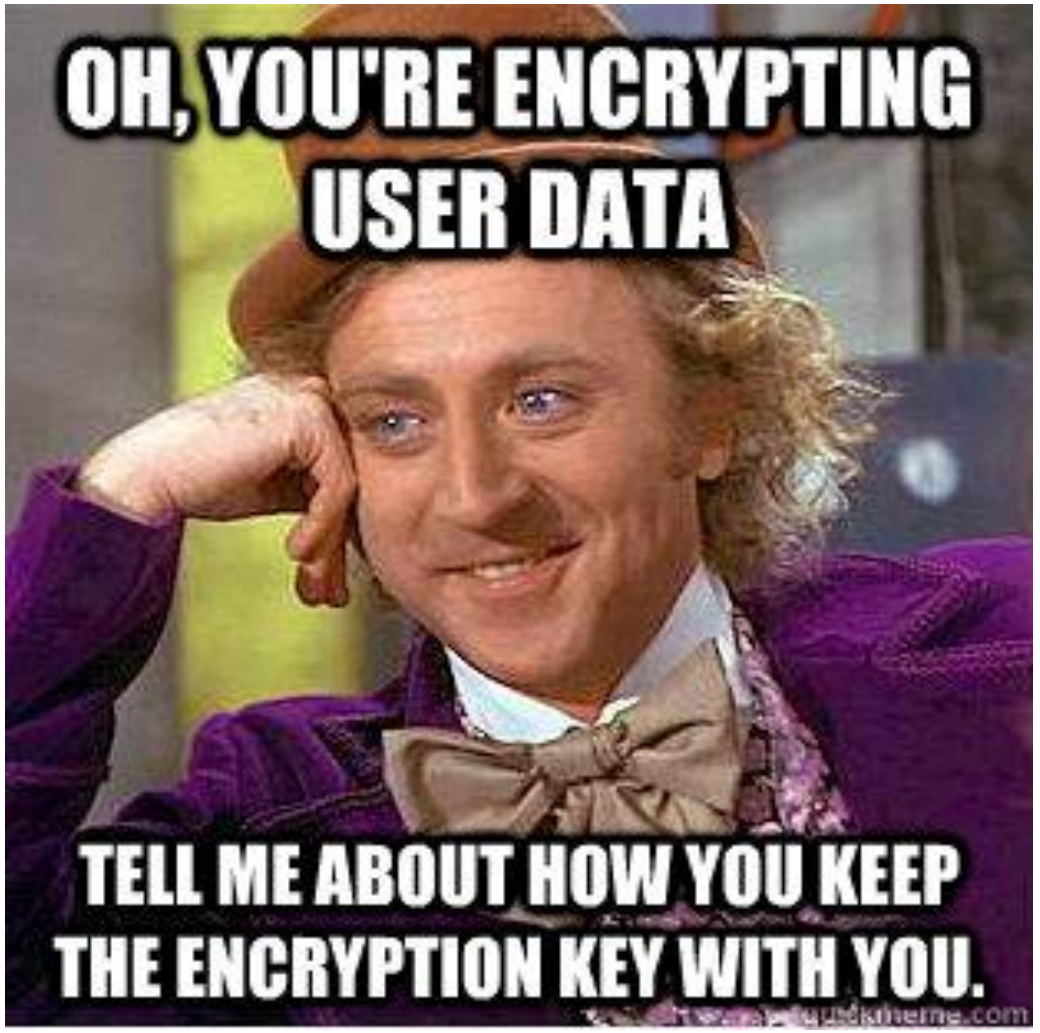
Principle of Least privilege - Software

Administrator account:

- Can do anything of that application?
- Do you allow him to LOGIN or access normal user's information?
- What is the use of the administrator account?
- Should you limit the power of the administrator?
  - The administrator can send out the "reset password" email, the email should send DIRECTLY to the user, but NOT the admin to set any password he likes

Principle of Least privilege – Development

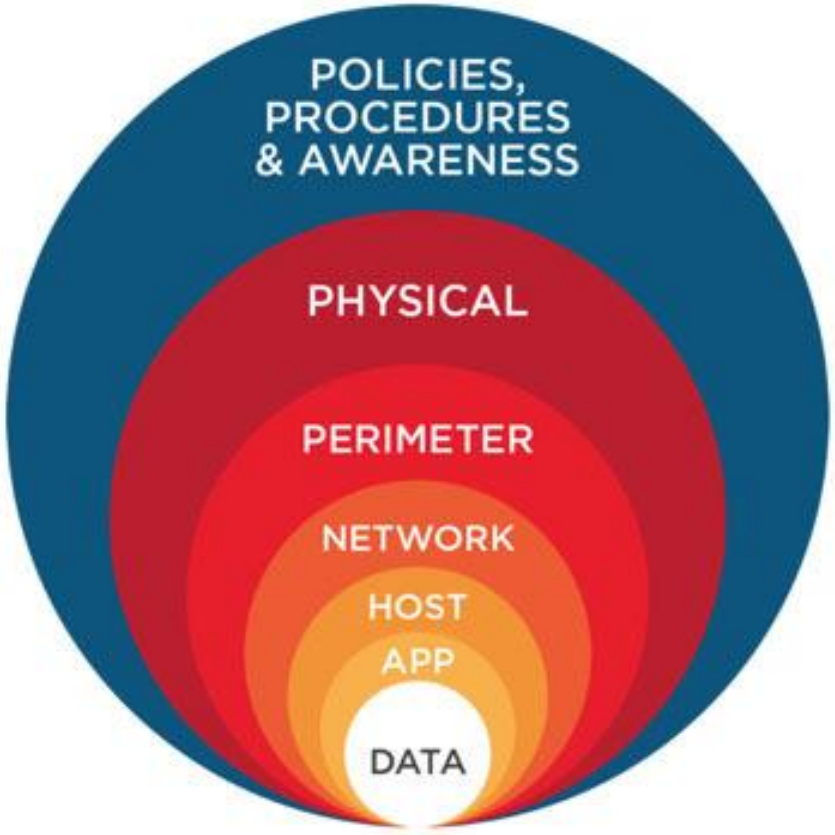
- Software Engineer:
  - Can he have ALL permission to access DB, the production and development environment?
    - Yes, or No?
- DB Administrator:
  - Can he have ALL information that is related to the users?
  - Yes, or No?



Principle of Defense in depth

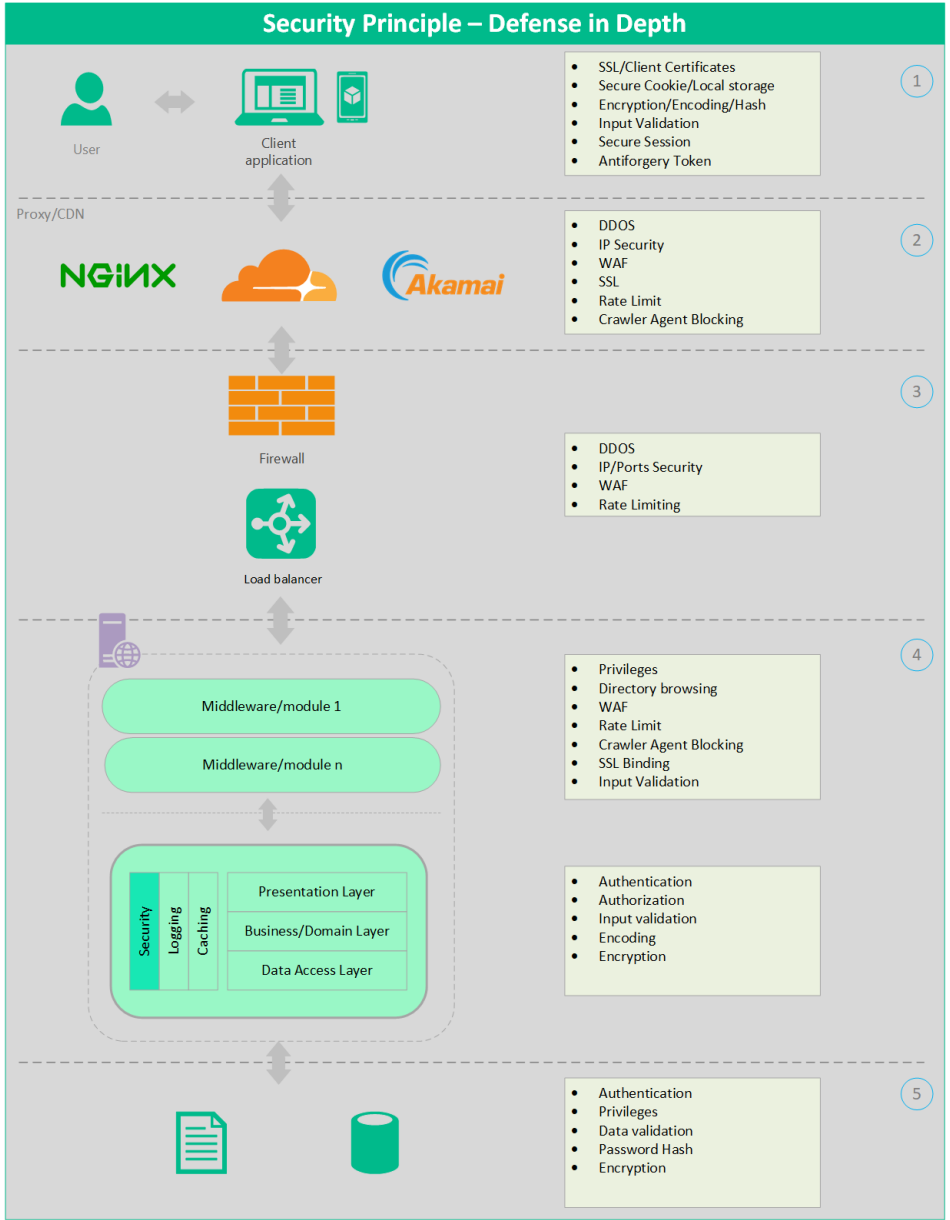
- The principle of **defense in depth** suggests that where one control would be reasonable, more controls that approach risks in different fashions are better. Controls, when used in depth, can make severe vulnerabilities extraordinarily difficult to exploit and thus unlikely to occur.

- With **secure coding**, this may take the form of tier-based validation, centralized auditing controls, and requiring users to be logged on all pages.



Principle of Defense in depth: An example

- A flawed administrative interface is unlikely to be vulnerable to anonymous attack if it **correctly gates access to production management networks**, checks for administrative user authorization, and **logs all access**.
- Thing in the way from client to your whole architecture!!!!
- Date flow diagram can also help to address the possible threats



Fail securely

- **Handling errors** securely is a key aspect of secure coding.
- There are **two types of errors** that deserve special attention.
  1. Exceptions that occur in the processing of a security control itself. It's important that these exceptions do not enable behavior that the countermeasure would normally not allow.
  2. As a developer, you should consider that there are generally **three possible outcomes** from a security mechanism:
     - allow the operation
     - disallow the operation
     - exception

Fail securely

Applications regularly fail to process transactions for many reasons. How they fail can determine if an application is secure or not.

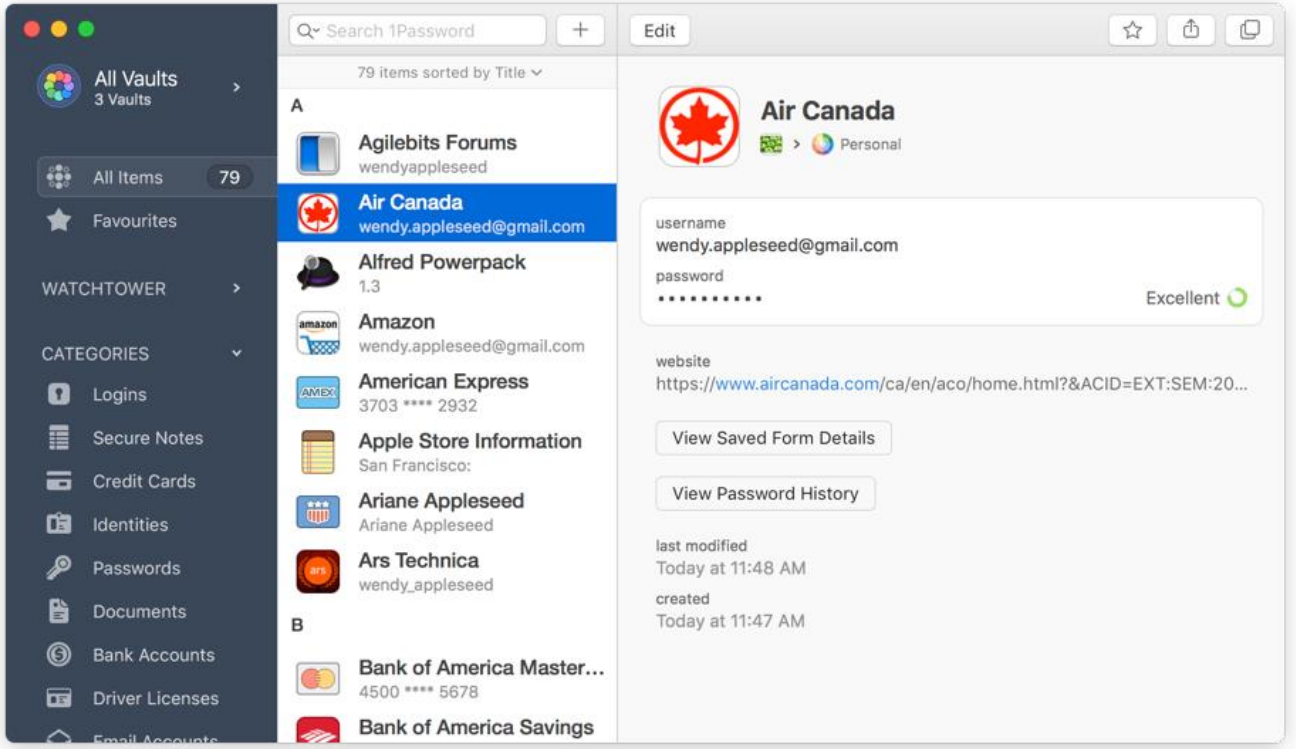
- For example:

```
isAdmin = true;
try {
        codeWhichMayFail();
        isAdmin = isUserInRole( "Administrator" );
}
catch (Exception ex)
{
        log.write(ex.toString());
}
```

How to fix this program?

Do not trust services

- Many organizations utilize the processing capabilities of third-party partners, who more than likely have differing security policies and posture than you.
- It is unlikely that you can influence or control any external third party, whether they are home users or major suppliers or partners.
- Therefore, implicit trust of externally run systems is not warranted.
- All external systems should be treated in a similar fashion.



Do not trust services

Major Cloudflare bug leaked sensitive data from customers' websites

Kate Conger @kateconger / 11:15 am +08 • February 24, 2017

Comment

Cloudflare revealed a serious bug in its software today that caused sensitive data like passwords, cookies, authentication tokens to spill in plaintext from its customers' websites. The announcement is a major blow for the content delivery network, which offers enhanced security and performance for more than 5 million websites.

This could have allowed anyone who noticed the error to collect a variety of very personal information that is typically encrypted or obscured.

Remediation was complicated by an additional wrinkle. Some of that data was automatically cached by search engines, making it particularly difficult to clean up the aftermath as Cloudflare had to approach Google, Bing, Yahoo and other search engines and ask them to manually scrub the data.

The leak may have been active as early as Sept. 22, 2016, almost five months before a security researcher at Google's Project Zero discovered it and reported it to Cloudflare.

Over 540 million Facebook records found on exposed AWS servers

Leak originated at two third-party companies that had collected Facebook data on their own servers.

By Catalin Cimpanu for Zero Day | April 3, 2019 -- 18:32 GMT (02:32 GMT+08:00) | Topic: Security

9

Data breach hunters have found two Amazon cloud servers storing over 540 million Facebook-related records that have been collected by two third-party companies. The number of affected users is believed to be in the range of millions and tens of millions.

SEE ALSO

10 dangerous app vulnerabilities to watch out for (free PDF)

Both servers have been discovered earlier this year by security researchers from UpGuard, a California-based cyber-security firm specialized in identifying data leaks.

LEAKS ORIGINATED AT THIRD-PARTY COMPANIES

The first server contained most of the data, and belonged to Cultura Colectiva, a Mexico-based online media platform operating across Spanish-speaking Latin America countries.

At a size of 146GB, this AWS server stored over 540 million records detailing user account names, Facebook IDs, comments, likes, reactions, and other data used for analyzing social media feeds and user interactions.

MORE FROM CATALIN CIMPANU

Security
Security researchers expose another instance of Chrome patch gapping

Security
Newly discovered cyber-espionage malware abuses Windows BITS service

Security
Cyber-security incident at US power grid entity linked to unpatched firewalls

Security
How to enable DNS-over-HTTPS (DoH) in Google Chrome

NEWSLETTERS

ZDNet Security

Your weekly update on security around the globe, featuring research, threats, and more.

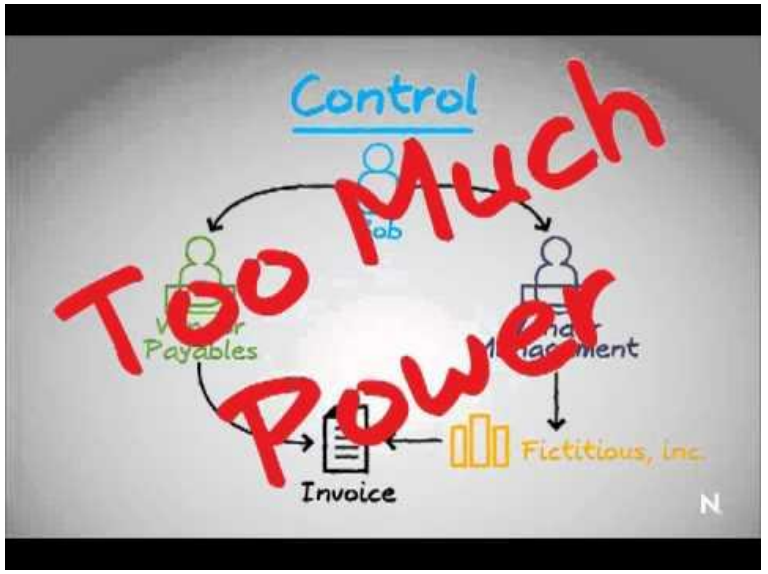
Your email address SUBSCRIBE

Using 3rd party service = secure?
Or just want to throw away the responsibilities?

Separation of duties

A key fraud control is **separation of duties**.

- For example, someone who requests a computer cannot also sign for it, nor should they directly receive the computer.
- This prevents the user from requesting many computers, and claiming they never arrived.



Certain roles have different levels of trust than normal users.

- **In particular, administrators are different from normal users**.
- In general, **administrators should not be users of the application**.

Separation of duties: An example

- An administrator should be able to turn the system on or off, set password policy
- An administrator shouldn’t be able to log on to the storefront as a super privileged user, such as being able to “buy” goods on behalf of other users.
- Take a look with this:
  - The admin have the permission to view / create user and to reset user’s password
  - He can login to the user account?
  - Backdoor?

Avoid security by obscurity

- Security through obscurity is a **weak security control**, and nearly always fails when it is the only control.

- This is not to say that keeping secrets is a bad idea, it simply means that **the security of key systems should not be reliant upon keeping details hidden**.

Avoid security by obscurity: An example

- The security of an application should not rely upon **knowledge of the source code being kept secret**.
- The security should rely upon many other factors, including reasonable password policies, defense in depth, business transaction limits, solid network architecture, and fraud and audit controls.
- A practical example is Linux. Linux's source code is widely available, and yet when properly secured, Linux is a hardy, secure and robust operating system.

Code audit?
White box?

Keep security simple

- Attack surface area and simplicity go hand in hand.
- Certain software engineering fads prefer **overly complex approaches** to what would otherwise be relatively straightforward and simple code.
- Developers should avoid the use of double negatives and complex architectures when a simpler approach would be faster and simpler.

Keep security simple: An example

- Although it might be fashionable to have a slew of singleton entity beans running on a separate middleware server, it is **more secure and faster to simply use global variables** with an appropriate mutex mechanism to protect against race conditions.

Fix security issues correctly

- Once a security issue has been identified, it is important to develop a test for it, and to understand the root cause of the issue.

- When design patterns are used, it is likely that the security issue is widespread amongst all code bases, so developing the right fix without introducing regressions is essential.

Fix security issues correctly: An example

- **A user has found that they can see another user's balance by adjusting their cookie**.
  - The fix seems to be relatively straightforward, but as the cookie handling code is shared among all applications, a change to just one application will trickle through to all other applications.
  - The fix must therefore be tested on all affected applications.

SINGAPORE INSTITUTE OF TECHNOLOGY

SECURITY ARCHITECTURE

Security architecture

- Applications without security architecture are as bridges constructed without finite element analysis and wind tunnel testing.

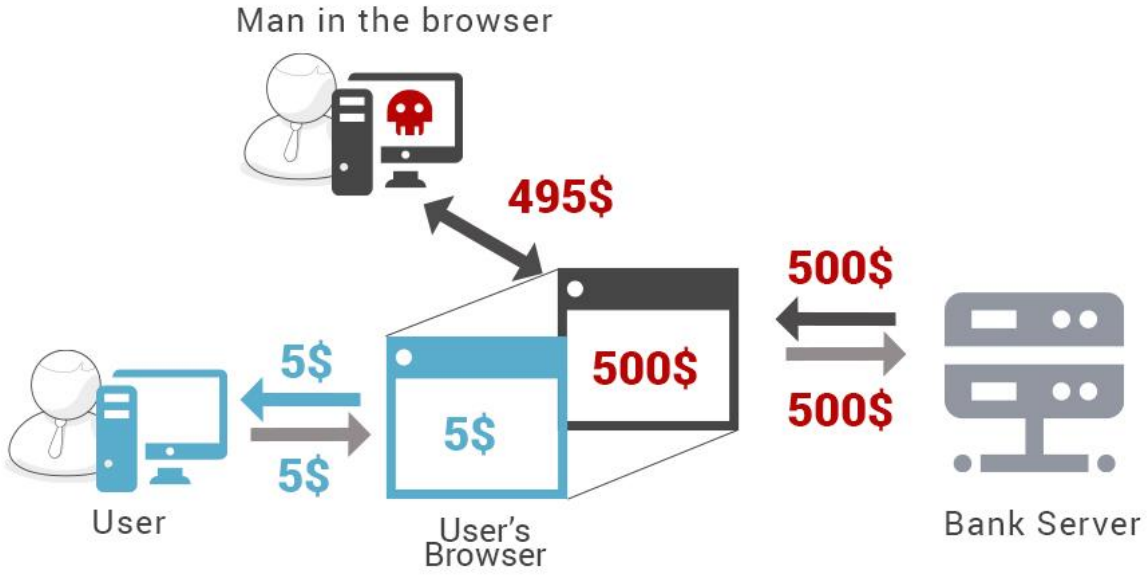
- They look like bridges, but they will fall down at the first flutter of a butterfly's wings.

- The need for application security in the form of security architecture is every bit as great as in building or bridge construction.

The london bridge is falling down code

Security architecture

- Application architects are responsible for constructing their design to adequately cover risks from both typical usage, and from extreme attack.
- Bridge designers need to cope with a certain number of cars and foot traffic but also cyclonic winds, earthquake, fire, traffic incidents, and flooding.
- Application designers must cope with extreme events, such as brute force or injection attacks, and fraud. The risks for application designers are well known.



Security architecture

- Security architecture refers to the fundamental pillars: the application must provide controls to protect the confidentiality of information, integrity of data, and provide access to the data when it is required (availability) – and only to the right users.

- Security architecture is not “markitecture”, where a cornucopia of security products are tossed together and called a “solution”, but a carefully considered set of features, controls, safer processes, and default security posture.

Security architecture

- Physical architecture
  - Provide a detailed, tangible representation of the system's physical infrastructure
  - Show the relationship among the application's physical components, distributed in different geographical locations
- Logical architecture
  - Emphasize the abstract, conceptual view of a system, illustrating the relationships and interactions between software components.